SalvageGraph Backend — Phase Breakdown

🟢 Phase 1 — Backend & Database Foundation

Status: COMPLETED ✅

```
Node.js
Express
TypeScript
Environment configuration
PostgreSQL
Prisma 7
Prisma Client
Database schema
Initial migration
```

Core database models:

```
User
Project
Material
MaterialImage
MaterialPassport
MaterialHistory
MaterialRequirement
Match
Recommendation
```

Your initial migration is complete.

🟡 Phase 2 — Backend Core Infrastructure

Status: NEXT

This phase makes the backend structurally reliable.

```
Phase 2
│
├── Prisma database service
├── Error handling
├── Async handler
├── API response structure
├── Request validation
├── Authentication middleware
```

```
├── JWT authentication
└── Basic API testing
```

Files:

```
src/
├── config/
│   ├── database.ts       ← NEXT
│   └── env.ts
│
├── middleware/
│   ├── auth.middleware.ts
│   ├── error.middleware.ts
│   └── validation.middleware.ts
│
├── utils/
│   ├── api-error.ts
│   └── async-handler.ts
│
└── validators/
```

Our immediate task: `database.ts`.

🟡 Phase 3 — Authentication & Users

```
Register
Login
Current user
JWT
Password hashing
Roles
Protected routes
```

Endpoints:

```
POST /api/auth/register
POST /api/auth/login
GET  /api/auth/me
```

Roles:

```
ADMIN
PROJECT_MANAGER
MATERIAL_MANAGER
```

```
INSPECTOR
USER
```

🟡 Phase 4 — Project Management

Build the project side of SalvageGraph.

```
Create Project
List Projects
View Project
Update Project
Delete Project
```

Endpoints:

```
POST   /api/projects
GET    /api/projects
GET    /api/projects/:id
PUT    /api/projects/:id
DELETE /api/projects/:id
```

🟡 Phase 5 — Material Inventory

This is the other major core module.

```
Register material
View materials
Update material
Delete material
Search/filter materials
Upload material images
```

Endpoints:

```
POST   /api/materials
GET    /api/materials
GET    /api/materials/:id
PUT    /api/materials/:id
DELETE /api/materials/:id
```

Initially, material information is manually entered.

Later:

```
Image
 ↓
```

```
AI
 ↓
Material type
Condition
Dimensions
Defects
```

🟡 Phase 6 — Material Passport

```
Material
   ↓
Passport
   ↓
QR Code
   ↓
Lifecycle history
```

Build:

```
POST /api/materials/:id/passport
GET  /api/materials/:id/passport
GET  /api/passports/:code
```

🟡 Phase 7 — Project Material Requirements

A project tells the system what it needs.

Example:

```
Project B
│
├── Brick → 10,000 pieces
├── Tile  → 500 pieces
└── Pipe  → 200 meters
```

Endpoints:

```
POST   /api/projects/:projectId/requirements
GET    /api/projects/:projectId/requirements
PUT    /api/requirements/:id
DELETE /api/requirements/:id
```

🟠 Phase 8 — Matching Engine

No AI yet.

First create a deterministic matching engine.

```
Material
    ↓
Hard filters
    │
    ├── Material type
    ├── Quantity
    ├── Dimensions
    ├── Condition
    └── Availability
    ↓
Candidate materials
    ↓
Suitability scoring
```

Then:

```
Match
 ↓
Score
 ↓
Explanation
```

This gives you a working matching system before ML.

🟠 Phase 9 — GIS & Logistics

```
Material location
       ↓
Project location
       ↓
Distance
       ↓
Transportation estimate
       ↓
Economic feasibility
```

Services:

```
gis.service.ts
cost.service.ts
```

Eventually:

```
PostGIS
OpenStreetMap
Routing
```

🟠 Phase 10 — Recommendation Engine

Combine:

```
Compatibility
Condition
Quantity
Dimensions
Distance
Transport cost
Economic benefit
```

into:

```
Suitability Score
       ↓
Ranked candidates
       ↓
Recommendation
```

Example:

```
Material M-102

Suitability: 86%

Reason:
✓ Material compatible
✓ Quantity sufficient
✓ Condition acceptable
✓ Dimensions compatible
✓ Transport feasible
✓ Economic benefit positive
```

🔵 Phase 11 — React Frontend

Once the APIs are stable:

```
Dashboard
Projects
```

```
Materials
Material Registration
Material Passport
Requirements
Matching
Recommendations
Maps
Verification
```

Architecture:

```
React
  ↓
Express API
  ↓
Services
  ↓
Prisma
  ↓
PostgreSQL
```

🔵 Phase 12 — AI Service

Now introduce the second backend.

```
Python
FastAPI
     │
     ├── YOLO
     ├── PyTorch
     ├── Scikit-learn
     └── XGBoost
```

Architecture:

```
React
  ↓
Node/Express
  ↓
AI Service
  ↓
Python/FastAPI
```

AI responsibilities can include:

```
Material identification
Defect detection
Condition assessment
Suitability prediction
```

🔵 Phase 13 — Human Verification

AI recommendation:

```
              AI
               ↓
        Recommendation
               ↓
        Human Inspector
          ↙     ↓      ↘
     Approve  Reject  Inspect
```

This is important because the project is intended to support reuse decisions, not automatically make safety decisions.

🔵 Phase 14 — Testing

```
Unit tests
Integration tests
API tests
Database tests
Matching tests
AI evaluation
```

We'll test things like:

```
Can 10,000 bricks satisfy a 7,000-brick requirement?
Can incompatible material types be rejected?
Does distance affect transport cost?
Does the recommendation explanation match the score?
```

🟣 Phase 15 — Docker & Deployment

Finally:

```
Docker Compose
│
├── frontend
├── backend
```

```
  ├── ai-service
  └── postgres
```

Then deployment.

Overall Progress

```
Phase 1   ████████████████████ 100% ✅
Phase 2   ░░░░░░░░░░░░░░░░░░░░   0% ← YOU ARE HERE
Phase 3   ░░░░░░░░░░░░░░░░░░░░   0%
Phase 4   ░░░░░░░░░░░░░░░░░░░░   0%
Phase 5   ░░░░░░░░░░░░░░░░░░░░   0%
Phase 6   ░░░░░░░░░░░░░░░░░░░░   0%
Phase 7   ░░░░░░░░░░░░░░░░░░░░   0%
Phase 8   ░░░░░░░░░░░░░░░░░░░░   0%
Phase 9   ░░░░░░░░░░░░░░░░░░░░   0%
Phase 10  ░░░░░░░░░░░░░░░░░░░░   0%
Phase 11  ░░░░░░░░░░░░░░░░░░░░   0%
Phase 12  ░░░░░░░░░░░░░░░░░░░░   0%
Phase 13  ░░░░░░░░░░░░░░░░░░░░   0%
Phase 14  ░░░░░░░░░░░░░░░░░░░░   0%
Phase 15  ░░░░░░░░░░░░░░░░░░░░   0%
```

The rule from now on

We finish one phase before moving to the next.

Right now, don't touch AI, React, Docker, or GIS.

We're starting **Phase 2 → Database Service**.

The next file we should implement is: